

به نام خدا

پروپوزال سامانه مدیریت و درخواست کارواش (Smart Carwash Service System)

اعضای تیم:

- حمیدرضا خداینده لو
- رضا عبداللهی
- محمد صادق بیاتی
- سامان اسماعیلی

مقدمه / بیان مسأله

امروزه بسیاری از شهروندان به دلیل مشغله‌های کاری، کمبود زمان و ترافیک، فرصت مراجعه حضوری به کارواش را ندارند. از طرف دیگر، صاحبان کارواش‌ها نیز به دنبال افزایش مشتری و مدیریت آسان خدمات خود هستند.

نبود یک سیستم یکپارچه برای ارتباط مشتریان با کارواش‌ها باعث بروز مشکلات زیر می‌شود:

- اتلاف زمان مشتری برای پیدا کردن کارواش مناسب
- عدم شفافیت قیمت‌ها و کیفیت کار
- عدم وجود فرآیند هماهنگ برای تحویل و بازگردانی خودرو
- فرصت از دست رفته برای کارواش‌ها جهت جذب مشتری بیشتر

این پروژه با هدف ایجاد سامانه‌ای هوشمند برای ثبت‌نام کارواش‌ها و فراهم کردن امکان درخواست خدمات توسط مشتریان به صورت تحویل در محل و بازگشت خودرو طراحی می‌شود.

ضرورت و اهمیت پروژه

چرا این پروژه مهم است؟

- افزایش راحتی و صرفه‌جویی در زمان برای مشتریان
- افزایش درآمد کارواش‌ها از طریق معرفی و سفارش آنلاین
- مدیریت و هماهنگی ساده فرآیند شستشو، تحویل و بازگردانی خودرو
- تأثیر بر جامعه / صنعت

- ارتقای کیفیت خدمات شهری
- ایجاد فرصت شغلی برای رانندگان حمل خودرو
- توسعه کسب‌وکارهای محلی با کمترین هزینه تبلیغات

هدف کلی سیستم

ایجاد یک سامانه کاربرپسند که مشتری، راننده و کارواش را به صورت یکپارچه به هم متصل کند و فرآیند درخواست، تحویل، شستشو و بازگردانی خودرو را خودکارسازی نماید.

چالش‌ها و محدودیت‌ها

- | | |
|---|-----------------------------|
| نیاز به سیستم امتیازدهی و احراز هویت | اعتماد مشتری در تحویل خودرو |
| نیاز به طراحی مناسب پایگاه داده و زمان‌بندی | مدیریت همزمان درخواست‌ها |

راهکارها

- ایجاد سیستم اعتبارسنجی و امتیازدهی برای راننده و کارواش
- پیاده‌سازی سیستم اعلان (Notification) برای هماهنگی بین طرفین

راه‌حل و امکانات پیشنهادی سیستم

سامانه تحت وب و اپلیکیشن موبایل با امکانات:

برای مشتری

- جستجو و مشاهده کارواش‌های نزدیک
- مشاهده قیمت، نظرات و امتیازات
- درخواست تحویل خودرو و ثبت سفارش شستشو

برای کارواش

- ثبت‌نام و تکمیل پروفایل
- مشاهده لیست سفارشات دریافتی
- مدیریت وضعیت شستشو و زمان تحویل

برای راننده حمل خودرو

• نمایش آدرس مشتری و کارواش

خروجی	فعالیت‌ها	بازه زمانی	فاز
سیستم احراز هویت آماده	تحلیل نیازمندی‌ها، طراحی دیتابیس، پیاده‌سازی Login/Signup	هفته 1-2	فاز 1
ماژول مدیریت کارواش	پیاده‌سازی ماژول کارواش (ثبت/ویرایش/مشاهده) + جستجوی کارواش	هفته 3-4	فاز 2
سیستم سفارش‌دهی کامل	پیاده‌سازی سیستم سفارش، راننده و وضعیت تحویل خودرو	هفته 5-6	فاز 3
نسخه نهایی قابل ارائه	پیاده‌سازی پرداخت، داشبورد مدیریتی، تست نهایی و مستندسازی	هفته 7-8	فاز 4

نمونه‌های مشابه

- CarwashToGo (ایران) – کارواش سیار بدون مدیریت تحویل خودرو
- WashOS (آمریکا) – سرویس شستشوی خودرو در محل بدون بخش راننده حمل خودرو

مزیت پروژه ما

- امکان تحویل خودرو و بازگردانی
- امتیازدهی راننده و کارواش
- مدل طراحی شده برای بازار ایران

نتیجه گیری

این سامانه با هدف بهبود تجربه مشتری، افزایش بهره‌وری کارواش‌ها و ایجاد ساختار حمل و هماهنگی استاندارد طراحی شده و قابلیت توسعه و تجاری‌سازی در سطح شهری و استانی را دارد.



Sprint 1 Backlog: The Foundation

Sprint Duration: 2 Weeks (Corresponds to Phase 1: Wk 1-2)

Sprint Goal:

"By the end of this sprint, we will **design and build** the core authentication and registration system. A new **Customer** will be able to register and log in. A new **Carwash** will be able to submit their application, and an **Admin** will be able to log in and approve or reject that application."

1. Foundational Design & Architecture Tasks

(To be completed in the first few days of the sprint)

- **[Task-F-Design] (Frontend Team - Figma Mockups)**
 - **Task:** Create high-fidelity Figma mockups for all screens in Sprint 1.
 - **Screens needed:**
 1. Customer Signup Screen
 2. Carwash Application Screen
 3. User Login Screen (handle all 3 roles)
 4. Admin Dashboard (showing the pending list)
 5. Customer Home (placeholder)
 6. Carwash Home (placeholder)
- **[Task-B-Design] (Backend Team - UML / Data Modeling)**
 - **Task:** Design and agree on the core data models for Sprint 1.
 - **Diagrams needed:**
 1. **UML Class Diagram:** Show the Customer, CarwashProfile, and Admin user models (e.g., how they relate to the base User model).
 2. **Entity Relationship Diagram (ERD):** Show the database tables and their relationships (e.g., User one-to-one with CarwashProfile).
- **[Task-API-Design] (Whole Team - API Contract)**
 - **Task:** Define and document the *exact* JSON request/response for all Sprint 1 API endpoints.
 - **Endpoints to define:**
 1. POST /api/customer/register/
 2. POST /api/carwash/apply/
 3. GET /api/admin/pending-carwashes/
 4. POST /api/admin/carwash/approve/<id>/
 5. POST /api/token/ (Login)

2. User Story Implementation & Tasks

User Story 1.1: Customer Signup

- **As a Customer**, I want to sign up with my email/phone so I can start using the service.
- **Acceptance Criteria (AC):**
 - (Given) I am on the Signup screen.
 - (When) I enter a valid email and password and tap "Sign Up."
 - (Then) My account is created in the database.
 - (And) I am automatically logged in and taken to the Customer dashboard.
 - (And) If my email is already taken, I see a clear error message.
- **Technical Tasks:**
 - **Backend (Django):**
 - [Task-B1] Create Customer User Model (e.g., inheriting from AbstractUser, or a one-to-one profile).
 - [Task-B2] Create CustomerRegistrationSerializer.
 - [Task-B3] Create API View (/api/customer/register/) to handle the POST request.
 - [Task-B4] Configure JWT or TokenAuthentication to return a token on successful registration.
 - **Frontend (Flutter):**
 - [Task-F1] Create RegistrationScreen.dart with email and password fields (based on Figma).
 - [Task-F2] Implement form validation (e.g., "is email valid?", "password long enough?").
 - [Task-F3] Create an AuthService to send the registration data to the /register/ endpoint (based on API Contract).
 - [Task-F4] On success, securely store the received token (e.g., using flutter_secure_storage).
 - [Task-F5] Navigate user to CustomerHomeScreen.dart (placeholder screen).

User Story 1.2: Carwash Registration Application

- **As a Carwash Owner**, I want to submit a registration application so I can be approved and listed on the platform.
- **Acceptance Criteria (AC):**
 - (Given) I am an unauthenticated visitor on the "For Businesses" page.
 - (When) I fill out the application form (Business Name, Location, Contact Person, etc.).
 - (Then) My application is saved in the database with a status of "Pending".
 - (And) I see a "Thank you for applying, we will review your application" message.
- **Technical Tasks:**
 - **Backend (Django):**
 - [Task-B5] Create CarwashProfile model with fields: business_name, location, contact_email, and a status field (e.g., 'Pending', 'Approved', 'Rejected').
 - [Task-B6] Create CarwashApplicationSerializer.

- [Task-B7] Create a public API View (/api/carwash/apply/) to handle the POST request (no auth required).
- **Frontend (Flutter):**
 - [Task-F6] Create CarwashApplicationScreen.dart with all the necessary form fields (based on Figma).
 - [Task-F7] Implement form validation for all required fields.
 - [Task-F8] Update AuthService to send the application data to the /apply/ endpoint (based on API Contract).
 - [Task-F9] On success, show a success dialog/pop-up message.

User Story 4.1: Admin Approves Carwash

- **As an Admin**, I want to see a queue of pending Carwash applications and be able to approve or reject them.
- **Acceptance Criteria (AC):**
 - (Given) I am logged in as an Admin.
 - (When) I navigate to the "Pending Applications" dashboard.
 - (Then) I see a list of all Carwashes with the "Pending" status.
 - (When) I click "Approve" on an application.
 - (Then) The Carwash status is changed to "Approved".
 - (And) A new, login-enabled User account is created for that Carwash.
- **Technical Tasks:**
 - **Backend (Django):**
 - [Task-B8] Create Admin-only API View (/api/admin/pending-carwashes/) that lists all profiles with status='Pending'.
 - [Task-B9] Create Admin-only API View (/api/admin/carwash/approve/<id>/) to handle the approval.
 - [Task-B10] **(Complex Task)** Write the business logic for the approval:
 1. Find the CarwashProfile by ID.
 2. Change its status to 'Approved'.
 3. Create a new User object (with a temporary password or linked to their email).
 4. Link the new User to the CarwashProfile.
 - **Frontend (Flutter):**
 - [Task-F10] Create AdminDashboard.dart (this will be the main screen after admin login, based on Figma).
 - [Task-F11] Create PendingCarwashList.dart widget that fetches data from /pending-carwashes/.
 - [Task-F12] Implement "Approve" and "Reject" buttons on each list item.
 - [Task-F13] On button press, call the appropriate admin API endpoint and refresh the list.

User Story 1.3: User Login (All Roles)

- **As any approved User (Customer, Carwash, Admin)**, I want to log in securely so I can access my specific panel.

- **Acceptance Criteria (AC):**
 - (Given) I am on the Login screen.
 - (When) I enter my correct credentials (email/password).
 - (Then) The system validates my credentials.
 - (And) I receive an authentication token.
 - (And) I am redirected to the correct dashboard (Customer, Carwash, or Admin) based on my role.
 - (And) If my credentials are wrong, I see a "Invalid username or password" error.
- **Technical Tasks:**
 - **Backend (Django):**
 - [Task-B11] Implement a global Login View (e.g., using djangorestframework-simplejwt's /api/token/ endpoint).
 - [Task-B12] **(Important)** Ensure the login response includes the user's **role** (e.g., "role": "Admin") so the frontend knows where to navigate (defined in API Contract).
 - **Frontend (Flutter):**
 - [Task-F14] Create LoginScreen.dart with email and password fields (based on Figma).
 - [Task-F15] Update AuthService to call the /api/token/ endpoint (based on API Contract).
 - [Task-F16] On success, store the token securely.
 - [Task-F17] **(Important)** Read the "role" from the API response and use a Navigator to push the correct screen (AdminDashboard.dart, CarwashDashboard.dart, or CustomerHomeScreen.dart).

Sprint 2 Backlog: "Open for Business"

Sprint Duration: 2 Weeks (Corresponds to Phase 2: Wk 3-4)

Sprint Goal:

"By the end of this sprint, the loop will be closed: An Admin approves a Carwash, that Carwash adds their services, and a Customer can find that Carwash in the app."

1. Carry-Over Items (High Priority)

(These are critical prerequisites for the new features)

User Story 4.1: Admin Approval Workflow

As an Admin, I want to view pending applications and click "Approve," **so that** the Carwash Owner gets their login credentials.

Acceptance Criteria (AC):

- (Given) I am logged in as an Admin on the "Pending Applications" screen.
- (When) I click the "Approve" button for a specific carwash.
- (Then) The carwash status updates to 'Approved' in the database.
- (And) A popup or modal displays the **generated password** for that new carwash owner (so the admin can share it).
- (And) The list refreshes and the approved carwash is removed from the "Pending" view.

Technical Tasks:

- **Backend (Django):**
 - [Task-B2.1] Update AdminCarwashApprovalView logic to explicitly return the generated temporary password in the API response JSON.
 - **Frontend (Flutter):**
 - [Task-F2.1] Create AdminPendingList.dart screen (fetching data from /api/carwash/admin/pending/).
 - [Task-F2.2] Implement the onApprove function to call the API.
 - [Task-F2.3] Create a SuccessDialog widget to display the returned email and password to the Admin.
-

User Story 2.1: Carwash Profile Completion

As a Carwash Owner, I want to update my working hours and location after logging in, **so that** my profile is accurate before customers see it.

Acceptance Criteria (AC):

- (Given) I am logged in as a Carwash Owner.
- (When) I navigate to the "My Profile" tab.
- (Then) I see my current details (Business Name, Address, Phone).
- (When) I edit my "Working Hours" or "Location" (Latitude/Longitude) and click Save.

- (Then) The changes are saved to the server and the view updates.

Technical Tasks:

- **Backend (Django):**
 - [Task-B2.2] Create CarwashProfileUpdateSerializer (allowing updates to working_hours, latitude, longitude, address).
 - [Task-B2.3] Create API View (PUT /api/carwash/profile/me/) that updates the logged-in user's carwash profile.
 - **Frontend (Flutter):**
 - [Task-F2.4] Create CarwashProfileScreen.dart with editable text fields.
 - [Task-F2.5] Integrate a Time Picker or Dropdown for "Working Hours" selection.
 - [Task-F2.6] Connect the "Save" button to the update API endpoint.
-

2. New Sprint 2 Items

User Story 2.4: Manage Carwash Services (The "Menu")

As a Carwash Owner, I want to add, edit, and delete services (e.g., "Basic Wash - \$10"), **so that** I have something to sell.

Acceptance Criteria (AC):

- (Given) I am logged in as a Carwash Owner on the "My Services" page.
- (When) I click "Add Service" and enter a Name, Description, and Price.
- (Then) The new service appears in my list.
- (When) I click "Delete" on an existing service.
- (Then) It is removed from the list.
- (Constraint) Price must be a positive number.

Technical Tasks:

- **Backend (Django):**
 - [Task-B2.4] Create CarwashServiceSerializer.
 - [Task-B2.5] Create API View (GET /api/carwash/services/) to list the logged-in owner's services.
 - [Task-B2.6] Create API View (POST /api/carwash/services/) to add a new service.
 - [Task-B2.7] Create API View (DELETE /api/carwash/services/<id>/) to remove a service.
 - **Frontend (Flutter):**
 - [Task-F2.7] Create ServiceManagementScreen.dart showing a list of services.
 - [Task-F2.8] Create a floating action button (+) that opens an AddServiceForm modal.
 - [Task-F2.9] Implement the API calls for List, Create, and Delete.
-

User Story 3.1: Customer Carwash Discovery

As a Customer, I want to see a list of all "Approved" carwashes, **so that** I can see who is available.

Acceptance Criteria (AC):

- (Given) I am logged in as a Customer on the Home Screen.
- (Then) I see a list of carwashes.

- (Constraint) The list **MUST NOT** show carwashes with status 'Pending' or 'Rejected'.
- (UI) Each list item shows the Business Name, Address, and Phone Number.

Technical Tasks:

- **Backend (Django):**
 - [Task-B2.8] Create CarwashListSerializer (showing public info only: name, address, location, hours).
 - [Task-B2.9] Create API View (GET /api/carwashes/) that returns CarwashProfile.objects.filter(status='APPROVED').
 - **Frontend (Flutter):**
 - [Task-F2.10] Create CustomerHomeScreen.dart.
 - [Task-F2.11] Build a CarwashCard widget to display individual carwash info nicely.
 - [Task-F2.12] Fetch the data from the API and map it to the list of cards.
-

Sprint 2 Deliverables Checklist

- [] Admin Panel: Working Approval Button.
- [] Carwash Panel: Profile Edit Screen.
- [] Carwash Panel: Add/Delete Service Screen.
- [] Customer App: Home Screen with List of Approved Carwashes.
- [] Backend: 4 New/Updated API Endpoints tested in Postman.

Sprint Backlog: Carwash Discovery & Booking

Sprint Duration: 2 Weeks

Sprint 3

Phase: Week 3–4



Sprint Goal

“By the end of this sprint, customers can search nearby carwashes, filter them, view full carwash profiles, and select services to begin the booking process.”

This sprint delivers the first fully functional version of the *Carwash Discovery Module*.

1. Carry-Over Prerequisites (High Priority)

These features are required before Epic 2 can function.

User Story 1.5 — Carwash Service Setup

As a Carwash Owner, I want to add/edit/delete my services so that customers can see what I offer.

Acceptance Criteria

- Logged-in carwash owners can add a new service (name, description, price).
- Added services appear instantly in the list.
- Owners can delete an existing service.
- Price must be a positive number.
- Changes are saved to the backend.

Technical Tasks

Backend (Django)

- [Task-B2.1] Create `CarwashServiceSerializer`.
- [Task-B2.2] Create API (GET `/api/carwash/services/`) – list services.
- [Task-B2.3] Create API (POST `/api/carwash/services/`) – add service.
- [Task-B2.4] Create API (DELETE `/api/carwash/services/<id>/`) – delete service.

Frontend (Flutter)

- [Task-F2.1] Create `ServiceManagementScreen.dart`.
 - [Task-F2.2] Implement modal dialog for adding services.
 - [Task-F2.3] Connect List / Add / Delete APIs.
-

User Story 2.1 — Search Nearby Carwashes

As a Customer, I want to search for carwashes near my location so I can find the closest one.

Acceptance Criteria

- User location is obtained (GPS or manual entry).
 - Carwashes are shown sorted by distance.
 - Each list item includes:
 - Name
 - Distance
 - Rating
 - Minimum service price
 - Results appear in both List View and Map View.
 - Loading time < 3 seconds.
 - “No results” message appears when needed.
-

Technical Tasks

Backend

- [Task-B2.5] Create API `/api/carwash/search/`.
- [Task-B2.6] Implement Haversine distance calculation.
- [Task-B2.7] Add ordering by distance.

- [Task-B2.8] Include name, rating, distance, min_price in response.
- [Task-B2.9] Write Unit Tests for search logic.

Frontend

- [Task-F2.4] Build `SearchScreen` (List + Map toggle).
 - [Task-F2.5] Integrate search API.
 - [Task-F2.6] Add loading states and error handling.
 - [Task-F2.7] Build `MapView` with location markers.
-
-

User Story 2.2 — Filter Carwashes

As a Customer, I want to filter carwashes by price, rating, and services so I can find the best match.

Acceptance Criteria

- Price range filter (min/max).
 - Rating filter (1–5 stars).
 - Service type filter (multi-select).
 - Multiple filters can be applied simultaneously.
 - Results refresh without full reload.
 - “No results after filtering” message displayed.
-

Technical Tasks

Backend

- [Task-B2.10] Add filter parameters to `/carwash/search`.
- [Task-B2.11] Implement price filtering.
- [Task-B2.12] Implement rating filtering.
- [Task-B2.13] Implement service-based filtering.
- [Task-B2.14] Write Unit Tests for filter logic.

Frontend

- [Task-F2.8] Build Filter UI (price slider, rating stars, service checkboxes).
- [Task-F2.9] Connect filters to search API.
- [Task-F2.10] Re-render search results dynamically.

User Story 2.3 — Carwash Profile View

As a Customer, I want to view a carwash's complete profile so I can compare and choose.

Acceptance Criteria

- Profile displays:
 - Name, address, working hours
 - List of services with prices
 - Ratings and user reviews
 - Carwash location on map
 - Loads fully in < 3 seconds.
 - "Select Services" button available.
-

Technical Tasks

Backend

- [Task-B2.15] Create API `/api/carwash/{id}/profile/`.
- [Task-B2.16] Include services, reviews, ratings, and basic info.
- [Task-B2.17] Unit Test for profile API.

Frontend

- [Task-F2.11] Create `CarwashProfileScreen`.
 - [Task-F2.12] Display services, reviews, and map location.
 - [Task-F2.13] Implement "Select Services" button.
-
-

User Story 2.4 — Service Selection (Booking Start – MVP)

As a Customer, I want to select services and begin the booking process.

Acceptance Criteria

- List of services fetched from backend.
 - Multi-select available.
 - Total price is calculated dynamically.
 - Selected services are saved as an Order Draft.
 - User is navigated to “Choose Time” (placeholder).
-

Technical Tasks

Backend

- **[Task-B2.18]** Create API `/api/order/prepare/`.
- **[Task-B2.19]** Save selected services in an OrderDraft model.
- **[Task-B2.20]** Validate selected services.

Frontend

- **[Task-F2.14]** Build Service Selection UI.
 - **[Task-F2.15]** Implement price calculation.
 - **[Task-F2.16]** Send selection to backend draft API.
 - **[Task-F2.17]** Navigate to next step (time scheduling placeholder).
-



Sprint 4 Backlog: "Operations & Logistics"

Sprint Duration: 2 Weeks (Phase 3: Wk 5-6)

Sprint Goal:

"By the end of this sprint, a Customer will be able to schedule a specific time for their service. The Carwash Owner will receive the order, assign a Driver, and update the status from 'Pending' to 'Completed'."

1. Booking Completion (High Priority)

Completing the flow started in Sprint 3.

User Story 2.4 (Part 2): Time Slot Selection

As a Customer, I want to choose a specific date and time for my carwash request so that I know when to expect the driver.

Acceptance Criteria (AC):

- [] (Given) I have selected services and clicked "Continue".
- [] (When) I am on the "Schedule" screen.
- [] (Then) I see a calendar or list of next 7 days.
- [] (When) I select a date, I see available time slots (e.g., 09:00, 10:00) based on the Carwash's `working_hours`.
- [] (Constraint) I cannot select a time in the past.
- [] (Constraint) I cannot select a slot that is already fully booked (Optional/Advanced: For MVP, assume infinite capacity or simple overlap check).

Technical Tasks:

- **Backend (Django):**
 - [Task-B4.1] Create API GET `/api/carwash/<id>/slots/?date=YYYY-MM-DD`. Logic: Return hours between open/close.
 - [Task-B4.2] Update `OrderPrepareView` or create `OrderFinalizeView` to accept `scheduled_time` and update the Draft order.
- **Frontend (Flutter):**
 - [Task-F4.1] Create `TimeSelectionScreen.dart` using a library like `table_calendar` or simple chips.
 - [Task-F4.2] Pass the selected time to the provider and call the update API.

User Story 2.5: Order Confirmation

As a Customer, I want to receive a confirmation summary after booking so I know my request was sent successfully.

Acceptance Criteria (AC):

- [] (When) I confirm the time and click "Book Now".
- [] (Then) The Order status changes from PENDING (Draft) to SUBMITTED.

- [] (And) I see a "Success" screen with the Order ID and estimated time.
- [] (And) I am redirected to my "Order History" or Home screen.

Technical Tasks:

- **Backend:**
 - [Task-B4.3] Update Order status transition logic. Ensure `scheduled_time` is saved.
 - **Frontend:**
 - [Task-F4.3] Create `BookingSuccessScreen.dart`.
-

2. Carwash Operations (The "Kitchen" Dashboard)

Giving the Carwash Owner the tools to work.

User Story 3.1 & 3.2: Manage Incoming Orders

As a Carwash Owner, I want to see a list of new orders and Accept or Reject them.

Acceptance Criteria (AC):

- [] (Given) I am logged in as a Carwash Owner.
- [] (Then) I see a dashboard tab "Orders" split into: *New*, *In Progress*, *Completed*.
- [] (When) I click on a *New* order (SUBMITTED), I see customer location and services.
- [] (Action) I can click **"Accept"** -> Status moves to ACCEPTED.
- [] (Action) I can click **"Reject"** -> Status moves to CANCELLED (Customer notified).

Technical Tasks:

- **Backend:**
 - [Task-B4.4] Create API GET `/api/carwash/orders/?status=...` (Filter by status).
 - [Task-B4.5] Create API POST `/api/carwash/orders/<id>/action/` (Body: `{"action": "accept"}` or `{"action": "reject"}`).
- **Frontend:**
 - [Task-F4.4] Create `CarwashOrderListScreen.dart` with Tabs (New, Active, History).
 - [Task-F4.5] Implement Accept/Reject buttons calling the API.

User Story 1.6 & 4.3: Driver Management (Logistics Setup)

As a Carwash Owner, I want to register my drivers so that I can assign them to orders.

Acceptance Criteria (AC):

- [] (Given) I am on the "Drivers" tab in the Carwash Panel.
- [] (When) I click "Add Driver", I enter Name, Phone Number, and Vehicle Plate.
- [] (Then) The driver is saved. (For MVP, we might skip the "Admin Approval for Drivers" step and just auto-approve them to save time, or keep it simple).

Technical Tasks:

- **Backend:**
 - [Task-B4.6] Create Driver model (linked to `CarwashProfile`).
 - [Task-B4.7] Create CRUD API `api/carwash/drivers/`.
- **Frontend:**

- [Task-F4.6] Create DriverManagementScreen.dart (List + Add Form).

User Story 3.3: Assign Driver

As a Carwash Owner, I want to assign a specific driver when accepting an order.

Acceptance Criteria (AC):

- [] (When) I click "Accept" on an order.
- [] (Then) A popup shows my list of available drivers.
- [] (When) I select a driver, the Order is updated with `driver_id` and status becomes `DRIVER_DISPATCHED`.

Technical Tasks:

- **Backend:**
 - [Task-B4.8] Update the "Accept" API (Task-B4.5) to accept a `driver_id` parameter.
 - **Frontend:**
 - [Task-F4.7] Add a Driver Selection Dropdown/Modal to the "Accept Order" flow.
-

3. Order Tracking Loop

Connecting the Customer and Carwash.

User Story 3.4: Order Status Updates

As a Carwash Owner, I want to update the status as the work progresses (Arrived, Washing, Done).

Acceptance Criteria (AC):

- [] (Given) An order is `DRIVER_DISPATCHED` (In Progress).
- [] (When) The work is done, I click "**Complete Order**".
- [] (Then) Status updates to `COMPLETED`.

Technical Tasks:

- **Backend:**
 - [Task-B4.9] Ensure status update API allows moving to `COMPLETED`.
- **Frontend:**
 - [Task-F4.8] Add "Complete" button to the *In Progress* order card.

User Story 3.7: Customer Order History

As a Customer, I want to see my active and past orders to track status.

Acceptance Criteria (AC):

- [] (Given) I am logged in as a Customer.
- [] (When) I go to "My Orders".
- [] (Then) I see my active booking with its current status (e.g., "Driver Dispatched").
- [] (And) I can see a history of past completed jobs.

Technical Tasks:

- **Backend:**
 - [Task-B4.10] Create API GET `/api/customer/orders/`.
 - **Frontend:**
 - [Task-F4.9] Create CustomerOrderListScreen.dart.
-

Sprint 4 Deliverables Checklist

1. ☐ **Customer:** Time Selection Screen & Order Success Screen.
2. ☐ **Customer:** My Orders Screen (Status tracking).
3. ☐ **Carwash:** Driver Management Screen.
4. ☐ **Carwash:** Order Management Dashboard (Accept/Reject/Complete).
5. ☐ **Backend:** 5+ New API endpoints operational.



Sprint 5 Backlog: Trust, Transactions & Polish

Sprint Duration: 2 Weeks (Corresponds to Phase 5: Wk 9-10)

Sprint Goal: "By the end of this sprint, the economic cycle is completed: Customers can pay for services and rate their experience. Carwash Owners can view their earnings and reputation. The Admin gains full control over the platform with dashboard metrics and user management."

1. Epic 5: Payments & Reviews (High Priority)

Focus: Closing the loop on orders with financial transactions and feedback.

User Story 5.1: Customer Payment

As a Customer, I want to pay for my service through the app after it's completed (or upfront), so that the order is finalized.

Acceptance Criteria (AC):

- (Given) An order status is "Completed" (or ready for payment).
- (When) I click "Pay Now".
- (Then) I am taken to a payment gateway (or a mock payment screen for MVP).
- (And) Upon success, the Order status updates to "Paid".
- (And) A transaction record is created.

Technical Tasks:

- **Backend (Django):**
 - [Task-B5.1] Create Transaction model (Order FK, Amount, Status, Timestamp).
 - [Task-B5.2] Create API POST /api/payment/initiate/ (Simulate bank connection).
 - [Task-B5.3] Create API POST /api/payment/verify/ to confirm payment and update Order status.
 - **Frontend (Flutter):**
 - [Task-F5.1] Create PaymentScreen.dart (Mock page with "Pay" button).
 - [Task-F5.2] Handle success response and navigate to "Rating Screen".
-

User Story 5.2 & 5.3: Ratings & Reviews

As a Customer, I want to rate the Carwash service (1-5 stars) and the Driver, so other users can know the quality.

Acceptance Criteria (AC):

- (Given) An order is "Paid" or "Completed".
- (When) The user is prompted to rate.
- (Then) They can assign stars (1-5) for the Carwash and stars (1-5) for the Driver.
- (And) They can add a text comment.
- (Then) The review is saved and linked to the Carwash/Driver profile.

Technical Tasks:

- **Backend (Django):**
 - [Task-B5.4] Create Review model (Carwash FK, Driver FK, User FK, Rating, Comment).
 - [Task-B5.5] Create API POST /api/reviews/submit/.
 - [Task-B5.6] Update CarwashProfile and Driver models to store/calculate average_rating.
 - **Frontend (Flutter):**
 - [Task-F5.3] Create ReviewDialog.dart widget (Star rating input).
 - [Task-F5.4] Trigger this dialog automatically after Payment success.
-

User Story 5.4 & 5.5: Owner Reputation Management

As a Carwash Owner, I want to see my average rating and reviews, and specific ratings for my drivers, so I can manage quality.

Acceptance Criteria (AC):

- (Given) I am a logged-in Carwash Owner.
- (When) I look at my Dashboard or "Reviews" tab.
- (Then) I see my overall average score (e.g., 4.5/5).
- (And) I can see a list of recent text reviews.
- (And) I can see individual ratings for each of my drivers.

Technical Tasks:

- **Backend (Django):**

- [Task-B5.7] Create API GET /api/carwash/reviews/ (List reviews for the owner).
 - [Task-B5.8] Create API GET /api/carwash/drivers/stats/ (Return driver names with their avg ratings).
 - **Frontend (Flutter):**
 - [Task-F5.5] Create ReviewsListScreen.dart for the owner panel.
 - [Task-F5.6] Add a "Rating" badge to the Driver List items.
-

User Story 5.6: Financials & Earnings

As a Carwash Owner, I want to view my total earnings and transaction history.

Acceptance Criteria (AC):

- (Given) I am on the Owner Dashboard.
- (Then) I see a "Total Earnings" card.
- (When) I click it, I see a history of completed, paid orders with their amounts.

Technical Tasks:

- **Backend (Django):**
 - [Task-B5.9] Create API GET /api/carwash/financials/ (Returns total revenue and list of transactions).
 - **Frontend (Flutter):**
 - [Task-F5.7] Create WalletScreen.dart or FinancialReport.dart.
 - [Task-F5.8] Integrate earnings data into the main Dashboard metrics.
-

2. Epic 4: Admin Validation & Platform Management (Remaining)

Focus: High-level control and user management.

User Story 4.1: Admin Dashboard Metrics

As an Admin, I want a dashboard to see key metrics (total users, total carwashes, total orders).

Acceptance Criteria (AC):

- (Given) I am logged in as Admin.
- (Then) The home screen shows cards: "Total Users", "Active Carwashes", "Total Orders Completed".

Technical Tasks:

- **Backend (Django):**
 - [Task-B5.10] Create API GET /api/admin/stats/ (Aggregated counts).
- **Frontend (Flutter):**
 - [Task-F5.9] Update AdminDashboard.dart to replace placeholders with real data charts/cards.

User Story 4.5: User Management

As an Admin, I want to manage all Customer accounts (e.g., reset passwords, ban users).

Acceptance Criteria (AC):

- (Given) I am an Admin.
- (When) I search for a user.
- (Then) I can Deactivate/Ban their account preventing further logins.

Technical Tasks:

- **Backend (Django):**
 - [Task-B5.11] Create API POST /api/admin/users/<id>/ban/.
- **Frontend (Flutter):**
 - [Task-F5.10] Create UserManagementScreen.dart (List of users + Ban button).

3. Epic 1: User Profiles (Remaining Item)

Focus: Enhancing the user experience.

User Story 1.4: Customer Vehicle Profile

As a Customer, I want to manage my profile and add my vehicle details (model, plate) for faster booking.

Acceptance Criteria (AC):

- (Given) I am a Customer in the "Profile" section.
- (When) I click "Add Vehicle".
- (Then) I can save my car's Model, Color, and Plate Number.
- (And) These details are auto-filled during the next booking.

Technical Tasks:

- **Backend (Django):**
 - [Task-B5.12] Create Vehicle model (User FK, Model, Color, Plate).

- [Task-B5.13] Create API CRUD /api/customer/vehicles/.
 - **Frontend (Flutter):**
 - [Task-F5.11] Create MyVehiclesScreen.dart.
 - [Task-F5.12] Integrate Vehicle selection into the "Order Placement" flow.
-

Sprint 5 Deliverables Checklist

- [] Customer App: Payment Screen & logic.
- [] Customer App: Rating & Review Dialog.
- [] Customer App: "My Vehicles" section.
- [] Carwash Panel: Reviews & Driver Ratings view.
- [] Carwash Panel: Earnings/Wallet screen.
- [] Admin Panel: Statistics Dashboard & User Ban functionality.